

CmpSc 160 – Distributed System

Final Project

1. Project Overview

Select from the ten (10) project titles (be sure each group doesn't have the same project title as other groups)

. The Distributed Concert Ticketing System (DCTS)

- **The Scenario:** A highly anticipated concert has limited seats. Multiple "Fan" clients connect to a "Box Office" server to book specific seats (e.g., "Seat A1").
- **The Synchronization Challenge:** Two fans try to book "Seat A1" at the exact same physical time. Lamport Clocks determine whose request actually originated first to award the ticket fairly.

2. The Decentralized Stock Exchange Simulator (DSES)

- **The Scenario:** "Trader" clients connect to an "Exchange" server. The server broadcasts the current price of a limited stock. Traders rush to send "Buy" orders.
- **The Synchronization Challenge:** If there are only 50 shares left and 100 shares are requested at the same time, the server uses Lamport timestamps to order the trades and distribute the remaining shares fairly.

3. The Distributed Trivia Buzzer System (DTBS)

- **The Scenario:** A "Host" server multicasts trivia questions to multiple "Player" clients. Players hit a key on their terminal to "buzz in" and answer.
- **The Synchronization Challenge:** Network lag might cause Player A's buzz to reach the server after Player B's, even if Player A physically pressed it first. The Lamport Logical Clocks will accurately order the buzzers so the true fastest player gets to answer.

4. The Distributed Ride-Hailing Dispatcher (DRHD)

- **The Scenario:** A "Dispatcher" server multicasts a high-paying ride request to multiple "Driver" clients scattered across the network. Drivers must press "Accept" to claim the ride.
- **The Synchronization Challenge:** The dispatcher must use Lamport Clocks to sort the incoming acceptance messages and award the fare to the driver who reacted first, notifying the losers that the ride was taken.

5. The Multiplayer Territory Claim Game (MTCG)

- **The Scenario:** A simple grid-based game (like a massive Tic-Tac-Toe board). "Player" clients connect to the "Game Master" server and send coordinates (e.g., X:4, Y:5) to claim a tile.
- **The Synchronization Challenge:** If two players try to claim the exact same tile simultaneously, the Game Master uses Lamport timestamps to decide who gets the tile and rejects the other player's move.

6. The Distributed Flash-Sale Storefront (DFSS)

- **The Scenario:** A "Store" server is selling a highly limited item (like an exclusive pair of sneakers) with only 3 units in stock. "Shopper" clients try to add the item to their cart.

- **The Synchronization Challenge:** Just like real-world bot traffic, the server will be flooded with purchase requests. It must order them strictly by their logical timestamps to ensure the first 3 shoppers get the item.

7. The Global Chat & Event Ordering System (GCEOS)

- **The Scenario:** A distributed chatroom where multiple "User" clients connect to a "Chat Server."
- **The Synchronization Challenge:** In standard chatrooms, network lag can cause messages to appear out of order, making conversations confusing. The server must use Lamport Clocks to sort incoming messages and multicast them back to the clients in the true chronological order they were typed.

8. The Distributed Cloud Resource Lock Manager (DCRLM)

- **The Scenario:** Simulating a datacenter. "Worker" clients connect to a "Lock Manager" server to request temporary, exclusive access to a shared resource (like a specific database or a GPU).
- **The Synchronization Challenge:** The server implements distributed mutual exclusion. It uses Lamport Clocks to place the workers in a queue. It grants the lock to the worker with the lowest timestamp, and the others must wait their turn.

9. The Decentralized Emergency Alert Network (DEAN)

- **The Scenario:** "Sensor" clients (acting as fire alarms) connect to a "Central Monitoring" server.
- **The Synchronization Challenge:** In an emergency, multiple sensors might trigger within milliseconds of each other. The server must use Lamport timestamps to determine the exact sequence of alarms to trace exactly where the fire started first, despite network delays in receiving the alerts.

10. The Distributed Peer-to-Peer Task Delegator (DPTD)

- **The Scenario:** A "Task Master" server broadcasts heavy computational jobs (e.g., "Calculate the 10,000th prime number") to multiple "Node" clients. Nodes race to claim the job.
- **The Synchronization Challenge:** Nodes send a "Claim" message back. The server uses Lamport Clocks to assign the job to the fastest node. Once assigned, the server multicasts a "Job Taken" message so other nodes don't waste power computing it.

2. Core Technical Requirements (The 3 Pillars)

To pass this final project, your system must successfully implement the three core topics of this course:

- **Pillar 1: Naming (The Directory Service)**
 - **Requirement:** Hardcoding IP addresses is strictly forbidden.
 - **Implementation:** You must build a **Naming Server** (acting like a DNS). When the Auctioneer server starts, it registers its physical IP/Port with the Naming Server under a structured name (e.g., auctio.server.main). When Bidders launch, they ask the Naming Server to resolve auction.server.main to an IP so they can connect.
- **Pillar 2: Message-Oriented Communication**

- **Requirement:** Communication must be asynchronous and handled via message passing.
- **Implementation:** Use Python sockets (Transient) or a lightweight Message-Oriented Middleware like ZeroMQ or RabbitMQ (Persistent). Bidders will send "Bid Messages" formatted in JSON. The Auctioneer must be able to broadcast "New Highest Bid" messages to all connected Bidders simultaneously (Multicast/Pub-Sub).
- **Pillar 3: Synchronization (Fair Event Ordering)**
 - **Requirement:** Bids must be ordered fairly, regardless of network delay.
 - **Implementation:** Implement **Lamport Logical Clocks**. Every node starts with a clock of 0. When a Bidder places a bid, it attaches its Lamport timestamp. If two bids arrive at the Auctioneer at the same physical time, the Auctioneer uses the Lamport timestamp to determine who *actually* bid first.

3. Team Member Roles (5 Members)

To ensure equal distribution of work, each member is assigned a specific role as the "Lead" for a component, though everyone will help with integration.

Member 1: The Registry Architect (Lead Naming Developer)

- **Tasks:** Build the naming_server.py. It must maintain a dictionary of logical names to physical IP/Ports.
- **Responsibilities:** Handle incoming registration requests from servers and lookup requests from clients. Ensure location independence (if the Auctioneer crashes and restarts on a new port, the Naming server must update seamlessly).

Member 2: The Middleware Engineer (Lead Communications Developer)

- **Tasks:** Design the underlying network communication layer.
- **Responsibilities:** Build the sockets/connections for sending and receiving messages. Define the JSON message structure (e.g., {"type": "bid", "amount": 50, "timestamp": 3}). Ensure the Auctioneer can successfully multicast messages to all connected clients without crashing.

Member 3: The Timekeeper (Lead Synchronization Developer)

- **Tasks:** Implement Lamport Logical Clocks across all nodes.
- **Responsibilities:** Write the Python functions that govern Lamport Rules 1, 2, and 3 (updating clocks on local events, sending, and receiving). You are responsible for the logic inside the Auctioneer that sorts incoming bids based on their logical timestamps to declare the true winner.

Member 4: The Client Developer (Lead Bidder UI/UX)

- **Tasks:** Build bidder_client.py.
- **Responsibilities:** Create an interactive terminal interface for the user. The client must dynamically ask the Naming Server for the Auctioneer's address, connect to it, allow the user to type in bid amounts, and display live updates (e.g., "Outbid! Current highest is \$100").

Member 5: The Server Developer & Integrator (Lead Auctioneer)

- **Tasks:** Build auction_server.py.

- **Responsibilities:** This is the core engine. You will integrate Member 2's networking and Member 3's clocks. The Auctioneer must register with Member 1's Naming server, accept multiple client connections, process the bids, determine the winner, and broadcast the results. You lead the final system testing.

4. Development Phases

Phase 1: Architecture & Setup

- Agree on the JSON data structures for your messages.
- Setup a shared GitHub repository.
- Member 1 builds and tests the standalone Naming Server.

Phase 2: Communication & Wiring

- Member 2 and Member 5 get the Auctioneer server running and accepting basic string messages from Member 4's Bidder client.
- Connect the Auctioneer and Bidder to the Naming Server to test name resolution.

Phase 3: Synchronization & Logic

- Member 3 integrates the Lamport Clock variables into the clients and the server.
- Implement the auction logic: setting a starting price, accepting bids for 60 seconds, sorting bids by Lamport timestamps, and declaring a winner.

Phase 4: Testing & Documentation

- Run the system on multiple different command prompts (or different laptops on the same Wi-Fi).
- Intentionally add `time.sleep()` to one of the bidder's code to simulate network lag, and prove that the Lamport Clocks still order the bids correctly.

5. Deliverables for Final Submission

1. **Source Code:** Well-commented Python files (`naming_server.py`, `auction_server.py`, `bidder_client.py`, `utils.py`).
2. **Architecture Diagram:** A visual map showing how the processes communicate (similar to the topologies discussed in your labs).
3. **Live Demonstration:** A 10-minute live demo showing the system working. The professor will ask you to start the naming server, start the auctioneer, and connect at least 3 bidders.
4. **Final Report (Docx):** * Introduction to the project.
 - How Naming was implemented.
 - How Message-Oriented Communication was handled.
 - How Lamport Logical Clocks solved the synchronization problem.
 - Individual reflections from each of the 5 members on what they learned.

6. Grading Rubric (100 Points)

- **Naming Implementation (20 pts):** Clients successfully use a logical name to find the server instead of hardcoded IPs.
- **Communication (20 pts):** Messages are passed successfully, without data loss, and the server can broadcast to all clients.
- **Synchronization (20 pts):** Lamport clocks are implemented correctly and govern the fairness of the auction.

- **System Stability (15 pts):** The system does not crash when multiple clients connect or bid simultaneously.
- **Documentation & Teamwork (15 pts):** Clear code comments, thorough final report, and evidence of equal work distribution.
- **Live Demo (10 pts):** Successful execution of the software during the final presentation.